ICT711 Programming and Algorithms Assessment 3 Project Report

1.0 Background

The ICT711 Assessment 3 requires students to collaboratively design, implement, and test a text-based Java application that demonstrates object-oriented programming principles, Java Collections, file handling, exception handling, and a command-line user interface [file:6]. The project specification also requires features such as adding, querying, updating, deleting, and periodically evaluating records stored in a text or CSV file containing at least 10 user records before execution [file:6].

The developed project is a book recommendation and library-member management system named E-UNIVERSE, designed around member records, reading activity, reward and penalty evaluation, and genre-based recommendation support [file:2][file:3][file:4][file:5]. The provided CSV file demonstrates the project's persistent data storage design using user records with fields for ID, name, email, membership type, books read, points, and preferred genre [file:1].

1.1 Introduction

The project was developed to provide an admin-controlled system for managing library members and supporting simple recommendation functionality in a structured Java environment [file:2][file:3]. The original implementation used a graphical interface, but the specification clearly requires a text-based application, which means the final compliant system is best presented as a command-line book recommendation and member management application [file:2][file:6].

At the core of the design is an abstract base class named `LibraryUser`, extended by `StandardMember` and `PremiumMember`, with different evaluation rules for each type of member [file:3][file:4][file:5]. This makes the system suitable for demonstrating abstraction, inheritance, polymorphism, and encapsulation in a single academic project [file:6].

1.2 Aim

The aim of this project is to design and implement a text-based Java application for managing library member records and providing simple book recommendations while satisfying the ICT711 requirements for object-oriented design, collections, file handling, exception handling, and CLI interaction [file:6][file:2].

1.3 Objectives

The objectives of the project are as follows:

- To create a Java application using multiple interacting classes and objects in a structured design [file:6].
- To use an abstract class and specialised subclasses to demonstrate abstraction and inheritance [file:3][file:4][file:5].
- To implement CRUD operations including add, view, query, update, and delete [file:2][file:6].
- To perform periodic member evaluation with rewards and penalties based on reading behaviour [file:4][file:5][file:6].
- To read from and write to a CSV file containing user data [file:1][file:2][file:6].
- To apply Java Collections, especially `ArrayList`, for in-memory data management [file:2][file:6].
- To handle invalid input and file errors using `try-catch` blocks [file:2][file:6].
- To provide the final system through a command-line interface consistent with the project specification [file:6].

1.4 Problem Statement

Managing member records manually is inefficient, especially when user information must be updated regularly, queried by different criteria, and stored persistently between program runs [file:6]. A library-related system also needs a mechanism to monitor reading activity and apply feedback, rewards, or penalties in a consistent manner [file:4][file:5][file:6].

The problem addressed by this project is the absence of a simple, maintainable Java-based system that can store member records, support administrative record management, provide recommendations, and automate member evaluation while following object-oriented design principles [file:2][file:3][file:6]. The project addresses this need through a text-based member management and recommendation system built with reusable classes and file-backed storage [file:1][file:6].

1.5 Scope of the Project

The scope of the project includes storing member records in a CSV file, loading them into memory, allowing an admin to manage those records, and saving updates back to file after operations are performed [file:1][file:2]. The implemented feature set includes user registration or insertion, viewing all records, querying by ID or name, updating values, deleting records, periodic evaluation, and recommendation output based on preferred genre [file:2][file:3][file:4][file:5].

The project is limited to a console-based local application and does not include database integration, networking, multi-user authentication, or advanced machine-learning recommendations [file:6]. The current data model focuses on member management rather

than a full real-world library catalogue, which keeps the system aligned with the academic scope of the assignment [file:3][file:6].

1.6 System Overview

The system is designed around a base class called `LibraryUser`, two member subclasses called `StandardMember` and `PremiumMember`, a source CSV file named `members.csv`, and a controller application class that performs all user operations [file:1][file:2][file:3][file:4][file:5]. The base class stores shared attributes such as ID, name, email, books read, points, and preferred genre, while each subclass implements its own evaluation logic [file:3][file:4][file:5].

In operation, the system loads member data from file into an `ArrayList`, provides a menu of actions to the admin, applies requested operations, and writes the modified records back to the CSV file [file:2][file:6]. This design supports the specification requirement for a structured, maintainable, text-based Java application using OOP and file handling [file:6].

2.0 Object-Oriented Programming (OOP) Concepts

The project specification requires the use of UML modelling, inheritance, polymorphism, abstraction, and other core object-oriented techniques in a Java application [file:6]. The developed system demonstrates these concepts through an abstract user hierarchy, specialised subclasses, shared method contracts, and class-based separation of responsibilities [file:3][file:4][file:5].

2.1 UML Diagrams

UML diagrams are required in the assessment to show the structure of classes and the relationships between them [file:6]. Based on the implemented code, the main diagram centres on the `LibraryUser` abstract class and its subclasses, while the controller class interacts with those objects and the CSV file [file:2][file:3][file:4][file:5].

2.1.1 Class Diagram

A suitable class diagram for this project should place `LibraryUser` as the abstract parent class containing common attributes and methods, with `StandardMember` and `PremiumMember` as derived subclasses implementing `evaluate()` differently [file:3][file:4][file:5]. The application controller class should be shown as managing a collection of `LibraryUser` objects and reading from or writing to `members.csv` [file:1][file:2].

Suggested UML text structure:

- `LibraryUser` (abstract): `id`, `name`, `email`, `booksRead`, `points`, `preferredGenre`, getters, setters, `evaluate()` [file:3]
- `StandardMember` extends `LibraryUser`: overridden `evaluate()` [file:4]
- `PremiumMember` extends `LibraryUser`: overridden `evaluate()` [file:5]
- `BookRecommendationSystemCLI` or main controller: `ArrayList<LibraryUser>`, load, save, add, query, update, delete, recommend, evaluate [file:2][file:6]

2.1.2 Relationships Between Classes

`StandardMember` and `PremiumMember` have a generalisation relationship with `LibraryUser` because they inherit common properties and behaviours from the abstract parent class [file:3][file:4][file:5]. The main controller has an association with `LibraryUser` objects because it stores and manages them inside an `ArrayList` during program execution [file:2].

The CSV file has an external data-source relationship with the controller because member records are read from the file during startup and written back during save operations [file:1][file:2]. This relationship is important because file persistence is one of the required components of the specification [file:6].

2.2 Inheritance

Inheritance is demonstrated by defining `LibraryUser` as a general abstract class and then extending it into `StandardMember` and `PremiumMember` classes [file:3][file:4][file:5]. This allows shared fields such as ID, name, email, books read, points, and preferred genre to be written once in the parent class and reused in the subclasses [file:3].

This design improves code reusability and reduces duplication, which directly supports the project requirement to design class hierarchies that promote reuse and extension for specific functionality [file:6]. The subclasses only need to implement the behaviour that differs, namely the periodic evaluation rules [file:4][file:5].

2.3 Polymorphism

Polymorphism is shown by storing both `StandardMember` and `PremiumMember` objects in a single collection of `LibraryUser` references [file:2][file:3][file:4][file:5]. When the program loops through users and calls `evaluate()`, each object performs the correct subclass-specific behaviour even though the reference type is the parent class [file:4][file:5].

This allows the system to treat multiple member types uniformly while still producing different outcomes for each type, which is exactly the behaviour described in the project specification [file:6]. Standard members receive a different point adjustment from premium members based on the same evaluation trigger [file:4][file:5].

2.4 Abstraction

Abstraction is demonstrated through the use of the abstract class `LibraryUser`, which defines common state and a general `evaluate()` method contract without specifying one single implementation for all users [file:3]. The implementation detail is deferred to the subclasses, where `StandardMember` and `PremiumMember` define their own evaluation behaviour [file:4][file:5].

This hides unnecessary detail from the rest of the program and supports cleaner design by focusing on what all users have in common at the abstract level [file:3][file:6]. The use of an abstract parent class matches the project brief's explicit reference to abstraction in system design [file:6].

2.5 Encapsulation

Encapsulation is implemented by making the `LibraryUser` attributes private and controlling access to them using getter and setter methods [file:3]. This prevents uncontrolled direct modification of fields and ensures that the class itself defines how its internal data is accessed and changed [file:3].

This approach improves maintainability and makes the class design more secure and structured, which is important in any multi-class object-oriented application [file:6]. Encapsulation also supports later expansion if additional validation logic needs to be added to setters or helper methods [file:3].

3.0 System Design

The system design combines an object-oriented user hierarchy, a file-based persistence layer, a Java Collections data structure, and a command-line user interface [file:2][file:3][file:4][file:5][file:6]. Its architecture is intentionally simple so that the implementation remains manageable within the assignment scope while still demonstrating the requested concepts [file:6].

3.1 System Architecture

The architecture follows a simple layered flow: the CSV file provides persistent data, the controller class loads it into memory, the collection stores active objects, the CLI presents options to the user, and output is saved back to file after changes [file:1][file:2][file:6]. The main internal entities are the member classes rather than separate user, admin, reader, and book objects, which means the current implementation is a focused member-management design rather than a full library-catalogue architecture [file:2][file:3].

A concise architecture description is:

- Input layer: `members.csv` file [file:1]
- Processing layer: controller logic for CRUD, recommendations, and evaluation [file:2]
- Domain layer: `LibraryUser`, `StandardMember`, `PremiumMember` [file:3][file:4][file:5]
- Interface layer: command-line menu aligned with the project brief [file:6]

3.2 Description of Classes

The implemented code contains a smaller class set than the section template suggests, so this report maps the requested headings to the actual project design where necessary [file:2][file:3][file:4][file:5]. Classes such as `Admin`, `Reader`, `Book`, `BookManager`, `RecommendationEngine`, and `FileManager` are not explicitly present in the original supplied code, so they are discussed here as logical roles handled by the main controller rather than as separate implemented files [file:2].

3.2.1 User Class

The closest implemented form of a user class is `LibraryUser`, which acts as the abstract parent for all member records [file:3]. It stores the core member information and defines the common interface used by the rest of the program [file:3].

3.2.2 Admin Class

A separate `Admin` class is not implemented in the supplied code [file:2]. Instead, the main controller class performs the administrative role by presenting options for adding, viewing, querying, updating, deleting, evaluating, and saving records [file:2][file:6].

3.2.3 Reader Class

A distinct `Reader` class is also not implemented in the current codebase [file:2][file:3]. The behaviour of a library reader is represented through member records held in `StandardMember` and `PremiumMember`, both of which track reading activity through the `booksRead` field [file:3][file:4][file:5].

3.2.4 Book Class

The current system does not include a dedicated `Book` entity because the recommendation feature works through genre-based output rather than a stored book catalogue [file:2]. Therefore, the system is better described as a member-centric recommendation tool rather than a complete book inventory system [file:2][file:6].

3.2.5 BookManager Class

A separate `BookManager` class is not part of the supplied implementation [file:2]. Its logical responsibilities, such as displaying recommendation-related output and handling user-facing operations, are combined in the main controller [file:2].

3.2.6 RecommendationEngine Class

A separate `RecommendationEngine` class is not explicitly present in the original code, although recommendation logic is included through a method in the controller [file:2]. In the improved version of the program, this logic can be described as genre-based rule processing that returns recommended titles according to the user's preferred genre [file:2][file:3].

3.2.7 FileManager Class

The current implementation does not define a standalone `FileManager` class [file:2]. File-reading and file-writing responsibilities are handled by methods such as `loadFromFile()` and `saveToFile()` inside the controller class, although separating them into a dedicated file manager would improve maintainability in a future version [file:2][file:6].

3.3 Attributes and Methods

The most important attributes in the system are `id`, `name`, `email`, `booksRead`, `points`, and `preferredGenre`, all stored in `LibraryUser` [file:3]. Important methods include getters and setters, `addPoints()`, `deductPoints()`, the abstract `evaluate()`, and controller-side methods for add, query, update, delete, recommend, load, and save operations [file:2][file:3][file:4][file:5].

Class	Key attributes or methods	Evidence
`LibraryUser`	`id`, `name`, `email`, `booksRead`, `points`, `preferredGenre`, getters, setters, `evaluate()`	[file:3]
`StandardMember`	Overridden `evaluate()` with +10 or -5 point logic	[file:4]
`PremiumMember`	Overridden `evaluate()` with +20 or -10 point logic	[file:5]
Controller	add, view, query, update, delete, evaluate, recommend, load, save	[file:2]

3.4 Data Flow of the System

The data flow begins when the program opens `members.csv` and reads user records into an in-memory `ArrayList` [file:1][file:2]. The admin then interacts with the system through the CLI to perform operations such as insert, query, update, delete, and evaluation [file:6].

After each major data-changing operation, the updated collection is written back to the CSV file so that the changes remain available in the next run of the program [file:2]. This read-process-write flow is consistent with the assignment's requirement for file-based storage and admin interaction after file loading [file:6].

4.0 System Functionality

The project specification lists required system functions including adding a user, querying user information, updating and deleting records, and overseeing periodic evaluation [file:6]. The developed system implements these features around member records and extends them with a recommendation output based on preferred genre [file:2][file:3].

4.1 User Registration

User registration is represented in this project by the add-user operation, where a new member record is created by entering ID, name, email, member type, books read, points, and preferred genre [file:2]. The controller then constructs either a `StandardMember` or `PremiumMember` object and stores it in the collection [file:2][file:4][file:5].

4.2 Adding a New Book

The current implementation does not contain a dedicated book catalogue or book-creation feature [file:2]. If this section must remain in the final template, it should be stated that the current project focuses on adding users rather than storing separate book entities, and recommendations are genre-based rather than title-database driven [file:2][file:6].

4.3 Viewing Records

Viewing records is supported through the ability to display all users and to search for selected users through query operations [file:2]. This makes it possible for the admin to inspect stored information before updating or deleting records [file:2][file:6].

4.3.1 Search by Book ID

The current code does not support search by book ID because the system manages users rather than stored book objects [file:2]. This heading can be adapted in the report to explain that the implemented equivalent is **search by user ID**, which is already provided in the query operation [file:2].

4.3.2 Search by Title

Search by title is not implemented because there is no dedicated `Book` class or book collection in the supplied system [file:2]. The nearest implemented search function is by user name [file:2].

4.3.3 Search by Author

Search by author is outside the scope of the current implementation because the program does not maintain author-based book records [file:2]. This section should be identified as a possible enhancement if the report template requires discussion [file:2].

4.3.4 Search by Genre

The data model does contain a `preferredGenre` field for each user, but the original code does not implement a separate query operation by genre [file:3][file:2]. Genre is currently used more directly for recommendation output than as a standalone search field [file:2][file:3].

4.4 Updating Records

The system supports record updating by allowing a selected user to be modified after locating the record by ID [file:2]. In the improved CLI version, the update behaviour is more complete because additional fields such as name, email, and preferred genre can also be changed rather than only books read and points [file:2][file:3].

4.5 Deleting Records

Deleting records is implemented through a user-ID-based delete operation, allowing the admin to remove a specific member from the collection [file:2]. The deletion is then saved back to the CSV file so that the removed record is no longer present in subsequent runs [file:2][file:1].

4.6 Book Recommendation Feature

The recommendation feature is implemented as a simple rule-based function that displays book suggestions to a selected user [file:2]. In the original GUI code, the same titles were shown for all members, but the improved version is better aligned with the available data because it uses the member's preferred genre to return more relevant titles [file:2][file:3].

4.7 Periodic Evaluation

Periodic evaluation is implemented through the `evaluate()` method in each subclass [file:4][file:5]. If a standard member has read more than five books, ten points are added; otherwise five points are deducted, while premium members receive twenty points or lose ten points based on the same threshold [file:4][file:5].

This behaviour directly matches the specification requirement for monitoring status, triggering positive and negative feedback, and applying reward or penalty actions to system entities [file:6]. It also provides one of the clearest demonstrations of runtime polymorphism in the system [file:4][file:5].

5.0 Utilising Java Collections

The project brief requires the use of Java Collections to manage system entities in memory [file:6]. The implemented code uses `ArrayList` for storing user objects, which is appropriate for the scale and type of operations performed in the system [file:2].

5.1 ArrayList Implementation

The main controller stores users in an `ArrayList<LibraryUser>`, allowing both `StandardMember` and `PremiumMember` objects to be managed in a single collection [file:2][file:4][file:5]. This supports sequential traversal, dynamic insertion, deletion, and repeated display operations during runtime [file:2].

5.2 Queue or LinkedList Usage

The current implementation does not actively use `Queue` or `LinkedList`, although the specification asks students to consider whether advanced data structures could provide benefits [file:6]. A `Queue` could be useful if recommendation or evaluation requests were processed in a first-in-first-out manner, while a `LinkedList` could be considered if frequent insertions and deletions in the middle of the collection became more important [file:6].

5.3 Justification for Collection Choice

`ArrayList` is justified because the system performs straightforward record management on a relatively small dataset loaded from a CSV file [file:1][file:2]. Its simple API, support for iteration, and direct fit with CRUD-style operations make it a suitable and understandable choice for this project [file:2][file:6].

For much larger systems, map-based or tree-based structures could improve search performance, but for this assignment the simplicity and readability of `ArrayList` are more

appropriate than introducing unnecessary complexity [file:6].

6.0 File Handling

File handling is a mandatory part of the assessment and is implemented through reading from and writing to a CSV file named `members.csv` [file:1][file:2][file:6]. The system uses this file as the persistent source of user records before and after admin operations [file:1][file:2].

6.1 Reading Data from Files

The system reads the CSV file during startup using buffered file input, skips the header row, splits each line into fields, and converts those fields into `LibraryUser` subclass objects [file:2]. This allows the application to begin with pre-existing data, which satisfies the requirement for at least 10 loaded user records before processing begins [file:1][file:6].

6.2 Writing Data to Files

After add, update, delete, or evaluation operations, the system writes the current collection back to `members.csv` using file output methods [file:2]. This ensures data persistence and allows modifications made during one session to be retained for future execution [file:2][file:6].

6.3 CSV/Text File Structure

The CSV file uses the header `id,name,email,memberType,booksRead,points,preferredGenre` and each row represents one user record [file:1][file:2]. This structure aligns with the attributes defined in `LibraryUser` and the member subclasses [file:1][file:3].

A sample row is shown below:

`1,Adesh Sharma,adesh@example.com,standard,12,75,Fiction` [file:1]

7.0 Exception Handling

The project specification requires the use of `try-catch` blocks for invalid user input and file-related errors [file:6]. The supplied code includes exception handling around user input, file loading, and file saving, although the improved CLI version strengthens this behaviour by avoiding silent failures and providing clearer feedback [file:2][file:6].

7.1 File Handling Exceptions

File handling exceptions occur when the program cannot open, read, or write the CSV file correctly [file:6]. The code addresses this through `try-catch` usage in file-loading and file-saving methods, which helps prevent the application from crashing immediately if file access fails [file:2].

7.2 Invalid Input Exceptions

Invalid input exceptions occur when the user enters values that cannot be parsed correctly, such as non-numeric IDs or other incorrect field values [file:2][file:6]. The program handles these cases by surrounding input-processing code with exception handling and prompting the user again or showing an error message [file:2].

7.3 Try-Catch Implementation

The original implementation uses `try-catch` blocks in add, query, update, delete, load, and save operations [file:2]. The report should note that, while exception handling is present, good practice requires more specific error messages and fewer empty catch blocks, which is why the improved CLI version strengthens error reporting and validation [file:2][file:6].

8.0 Command-Line User Interface (CLI)

The assignment requires a text-based user interface, specifically a command-line environment where the admin can choose operations from a menu [file:6]. Although the original code was written with Swing, the corrected final version is best documented as a CLI system so that it satisfies the specification [file:2][file:6].

8.1 Main Menu Design

The main menu presents the available operations in a numbered list so the admin can choose an action clearly and consistently [file:6]. The menu includes add user, view all users, query user, update user, delete user, run periodic evaluation, recommend books, and save and exit, which aligns with the system's implemented feature set [file:2].

8.2 User Interaction Flow

The interaction flow begins with file loading, after which the user is shown the menu and prompted to select an option [file:1][file:6]. Depending on the choice, the program either collects new input, searches the collection, updates existing records, deletes records, performs evaluation, or displays recommendations before returning to the menu [file:2].

8.3 Menu Navigation

Menu navigation is linear and easy to follow, which is suitable for an academic command-line application [file:6]. After each operation, the program returns to the menu until the user chooses the exit option, at which point the file is saved and the system closes [file:2].

9.0 Testing and Program Output

Testing is necessary to verify that each required feature behaves correctly and that the system responds safely to invalid conditions [file:6]. The most relevant tests for this project involve file loading, CRUD operations, evaluation rules, and invalid input handling [file:2][file:4][file:5].

9.1 Test Cases

The following table presents suitable test cases for the system:

Test case	Input or action	Expected result	Source
Load CSV file	Start program with valid `members.csv`	Users are loaded into memory	[file:1] [file:2]
Add valid user	Enter a new unique ID and valid details	New member is stored and saved	[file:2]
Add duplicate ID	Enter an ID already present in file	System rejects insertion in improved version	[file:1] [file:2]
Query by ID	Enter an existing ID	Matching user details are displayed	[file:2]
Query by name	Enter part of a stored name	Matching user details are displayed	[file:2]
Update user	Enter valid modified values	Selected record changes and saves	[file:2] [file:3]
Delete user	Enter an existing ID	Record is removed and file updates	[file:2]
Run evaluation	Trigger evaluation option	Points are adjusted per member type	[file:4] [file:5]
Invalid file	Missing or inaccessible CSV file	Error is handled without uncontrolled crash	[file:2] [file:6]

9.2 Program Screenshots

Program screenshots should be captured from the CLI while demonstrating major features such as menu display, add user, query result, update confirmation, delete confirmation, evaluation output, and recommendation output [file:6]. These screenshots should be inserted in the final report after the test-case discussion to provide visual evidence of execution [file:6].

9.3 Output Verification

Output verification should confirm that the system produces correct user records, appropriate recommendations, and correct point changes after evaluation [file:2][file:4][file:5]. Verification should also confirm that file updates are reflected in `members.csv` after operations are completed [file:1][file:2].

10.0 Conclusion

The developed project satisfies the main goals of the ICT711 Assessment 3 by demonstrating object-oriented design, CRUD functionality, Java Collections usage, file handling, exception handling, and a text-based user interface design for a library-member recommendation context [file:6][file:2][file:3][file:4][file:5]. The use of `LibraryUser`, `StandardMember`, and `PremiumMember` provides a clear demonstration of inheritance, polymorphism, abstraction, and encapsulation in a manageable academic system [file:3][file:4][file:5].

The project is intentionally limited in scope, but it successfully addresses the required features while remaining extensible for future enhancement, such as adding separate book classes, a dedicated file manager, and richer recommendation logic [file:2][file:6]. This makes the system appropriate for the learning outcomes of the assessment and suitable for group project documentation [file:6].

11.0 Java Source-Code Link

The project specification states that the project report should provide a drive or Git repository link for the Java source code and source data file [file:6]. This section should be completed by inserting the group's actual Google Drive or GitHub repository URL before final submission [file:6].

Example format:

```
Source Code Repository Link: https://github.com/your-group-name/ict711-project
```

12.0 References

- ICT711 Programming and Algorithms Assessment 3 specification document [file:6]
- `BookRecommendationSystemGUI.java` source file [file:2]
- `LibraryUser.java` source file [file:3]
- `StandardMember.java` source file [file:4]
- `PremiumMember.java` source file [file:5]
- `members.csv` data file [file:1]

12.0 Appendix

Appendix material may include the UML class diagram, CLI screenshots, selected code excerpts, test-output evidence, and the final cleaned CSV structure [file:1][file:6]. This section is also a suitable place to include additional design notes, group-member data records, or implementation snapshots that would interrupt the main report flow if inserted earlier [file:6].